

Performance Testing Plan & Report

Focused performance checks for the incident automation workflow

DATE	STUDENT NAME	PROJECT
07 October 2026	Gangadevi.R	Incident Life Cycle Automation

1. Test objectives

- Verify that common incident actions remain responsive under expected project-level load.
- Check that scheduled SLA monitoring does not block user-facing requests.
- Confirm that incident search/filtering remains usable as records grow.

2. Performance test scenarios

ID	Scenario	Test Input	Measure	Expected
PT-01	Create incident	Submit a valid incident with required fields	Measure end-to-end response and successful creation	Expected: acceptable interactive response under expected load
PT-02	Open incident detail	Retrieve a single incident and lifecycle history	Measure read response time	Expected: stable response with normal history size
PT-03	Search & filter	Query by status, priority, owner and date	Measure query latency	Expected: predictable response with indexed fields
PT-04	SLA background check	Process due / at-risk incidents in scheduled job	Measure job duration and user-request impact	Expected: background processing does not materially block UI/API
PT-05	Concurrent updates	Multiple users update separate incidents	Observe errors and throughput	Expected: no lost updates or unexpected failures

3. Execution record

Test ID	Environment	Observed Result	Status	Notes
PT-01	Project test environment	To be recorded during execution	Pending	Capture actual timings and resource usage
PT-02	Project test environment	To be recorded during execution	Pending	Capture actual timings and resource usage
PT-03	Project test environment	To be recorded during execution	Pending	Capture actual timings and resource usage

Test ID	Environment	Observed Result	Status	Notes
PT-04	Project test environment	To be recorded during execution	Pending	Capture actual timings and resource usage
PT-05	Project test environment	To be recorded during execution	Pending	Capture actual timings and resource usage

4. Acceptance guidance

Use measured response times from the actual implementation in the final submission. Avoid inventing benchmark numbers before execution.